Add an option, gen_data --edge <out_dir>, that writes these small files instead: an empty file; a file with 5 values; a file with exactly 7 values; a file containing the token "abc"; and a flat file with noise but no pulses. Don't change anything else.

Add an early `--edge` mode before the existing four-argument usage check. Leave the normal generation code unchanged.

The command will be:

None

```
./gen_data --edge test_edges
```

It creates five files:

- `empty.dat` — zero values.
- `five.dat` — exactly 5 integer values.
- `seven.dat` — exactly 7 integer values.
- `invalid.dat` — contains `abc`.
- `noise_only.dat` — Gaussian noise with no pulses.

Code to insert

In `gen_data.cpp`, insert this immediately after:

None

```
int main(int argc, char* argv[]) {
```

None

```
    // Edge-case test file generation mode.
    if (argc == 3 && std::string(argv[1]) == "--edge") {
        fs::path out_dir = argv[2];
        fs::create_directories(out_dir);

        // 1. Empty file.
```

```
        std::ofstream(out_dir / "empty.dat");

        // 2. Exactly 5 values.
        {
            std::ofstream out(out_dir / "five.dat");
            out << "10\n-20\n30\n-40\n50\n";
        }

        // 3. Exactly 7 values.
        {
            std::ofstream out(out_dir / "seven.dat");
            out << "10\n-20\n30\n-40\n50\n-60\n70\n";
        }

        // 4. Invalid non-integer token.
        {
            std::ofstream out(out_dir / "invalid.dat");
            out << "10\n20\nabc\n30\n";
        }

        // 5. Noise-only file, no pulses.
        {
            std::ofstream out(out_dir / "noise_only.dat");
            std::mt19937 edge_rng(12345);
            std::normal_distribution<double> noise(0.0, 20.0);

            for (int i = 0; i < 100; ++i) {
                out <<
static_cast<int>(std::llround(noise(edge_rng)))
                    << '\n';
            }
        }
```

```
        std::cout << "Generated edge-case files in "
                  << out_dir << '\n';
        return 0;
    }
```

Compile and test

```
None
g++ -std=c++23 -Wall -Wextra -pedantic gen_data.cpp -o
gen_data
./gen_data --edge edge_tests
```

Expected output directory:

```
edge_tests/
├── empty.dat
├── five.dat
├── seven.dat
├── invalid.dat
└── noise_only.dat
```

Your original four-argument generation mode, seed behavior, pulse generation, and `truth.txt` output remain unchanged.

Testing tip: These files let you check how your pulse detector handles empty input, insufficient samples, exactly seven samples, invalid input, and noise without actual pulses.

Write a short header, stage_timer.h, for a C++23 program. It should define an RAII class, StageTimer, that I put at the top of a block of code:
 StageTimer t{"smooth"};
When t is destroyed, add the elapsed time, measured with std::chrono::steady_clock, to a running total for that name.
Also write print_stage_report(), which prints each name's total milliseconds and call count to std::cerr -- not std::cout, so my program's normal output does not change. Standard library only,

under 60 lines, and explain each part.

Here is a standalone, header-only C++23 implementation of `stage_timer.h`. It uses `std::chrono::steady_clock` for timing and stores accumulated time and call counts by stage name.

`stage_timer.h`

None

```
#pragma once

#include <chrono>
#include <iostream>
#include <map>
#include <string>

class StageTimer {
    using Clock = std::chrono::steady_clock;

    struct Result {
        double milliseconds = 0.0;
        std::size_t calls = 0;
    };

    inline static std::map<std::string, Result> totals{};

    std::string name;
    Clock::time_point start;

public:
    explicit StageTimer(std::string stage_name)
        : name(std::move(stage_name)), start(Clock::now()) {}

    ~StageTimer() {
```

```
        auto elapsed = std::chrono::duration<double,
std::milli>(
            Clock::now() - start
        ).count();

        auto& result = totals[name];
        result.milliseconds += elapsed;
        ++result.calls;
    }

    StageTimer(const StageTimer&) = delete;
    StageTimer& operator=(const StageTimer&) = delete;

    static void print_stage_report() {
        std::cerr << "\nStage timing report:\n";

        for (const auto& [name, result] : totals) {
            std::cerr << name
                      << ": " << result.milliseconds << " ms,
"
                      << result.calls << " calls\n";
        }
    }
};
```

How to use it

Include the header:

```
None
#include "stage_timer.h"
```

Place a timer at the beginning of each block you want to measure:

None

```
void process_data() {
    {
        StageTimer t{"smooth"};
        // Smoothing code
    }

    {
        StageTimer t{"detect"};
        // Pulse detection code
    }

    {
        StageTimer t{"smooth"};
        // Another smoothing operation
    }
}

int main() {
    process_data();

    StageTimer::print_stage_report();
}
```

Example output (on `stderr`):

```
Stage timing report:
detect: 2.31 ms, 1 calls
smooth: 4.82 ms, 2 calls
```

Explanation

1. `steady_clock`: Measures elapsed time using a monotonic clock, which is appropriate for performance measurements.

2. **Constructor:** Saves the stage name and records the starting time.
3. **Destructor:** Calculates elapsed milliseconds and adds them to the corresponding entry in `totals`.
4. **Static map:** Shares accumulated timing information across all `StageTimer` objects.
5. **Call counter:** Increments every time a timer with that name is destroyed.
6. **Deleted copy operations:** Prevent accidental copying, which could otherwise cause incorrect timing totals.
7. `print_stage_report()`**:** Prints accumulated milliseconds and call counts to `std::cerr`, leaving normal program output untouched.

One important detail: The timer should be scoped around the code you want to measure. Its destructor runs automatically when the block ends, including when the block exits early.

The header uses only the C++ standard library and is suitable for including in multiple source files.

About suggestion <1>: my timing report says that stage takes <90>% of the run time. Is the change worth making? What does the compiler already do at -O2? What measurement would show whether

it helps?

Yes, suggestion #1 is worth testing, but I wouldn't commit to changing your code until you've measured the improvement.

Your timing results show that reading is the biggest bottleneck. However, replacing `std::istream_iterator<int>` with `std::from_chars` introduces more complexity, so the performance gain needs to justify that complexity.

1. Why the 90% matters

Your reported reading time is 489.714 ms out of approximately 543.49 ms of measured stage time.

Reading


90.1%

All other measured stages


9.9%

That means improving reading has much more potential than optimizing smoothing or piggyback detection.

For example, assuming the other stages remain unchanged:

These are hypothetical results, not predicted gains.

Also, 90% refers to the **sum of your timed stages**, not necessarily the complete program's runtime. Directory traversal, file opening, and some output operations fall outside those measurements.

2. What does `-O2` already optimize?

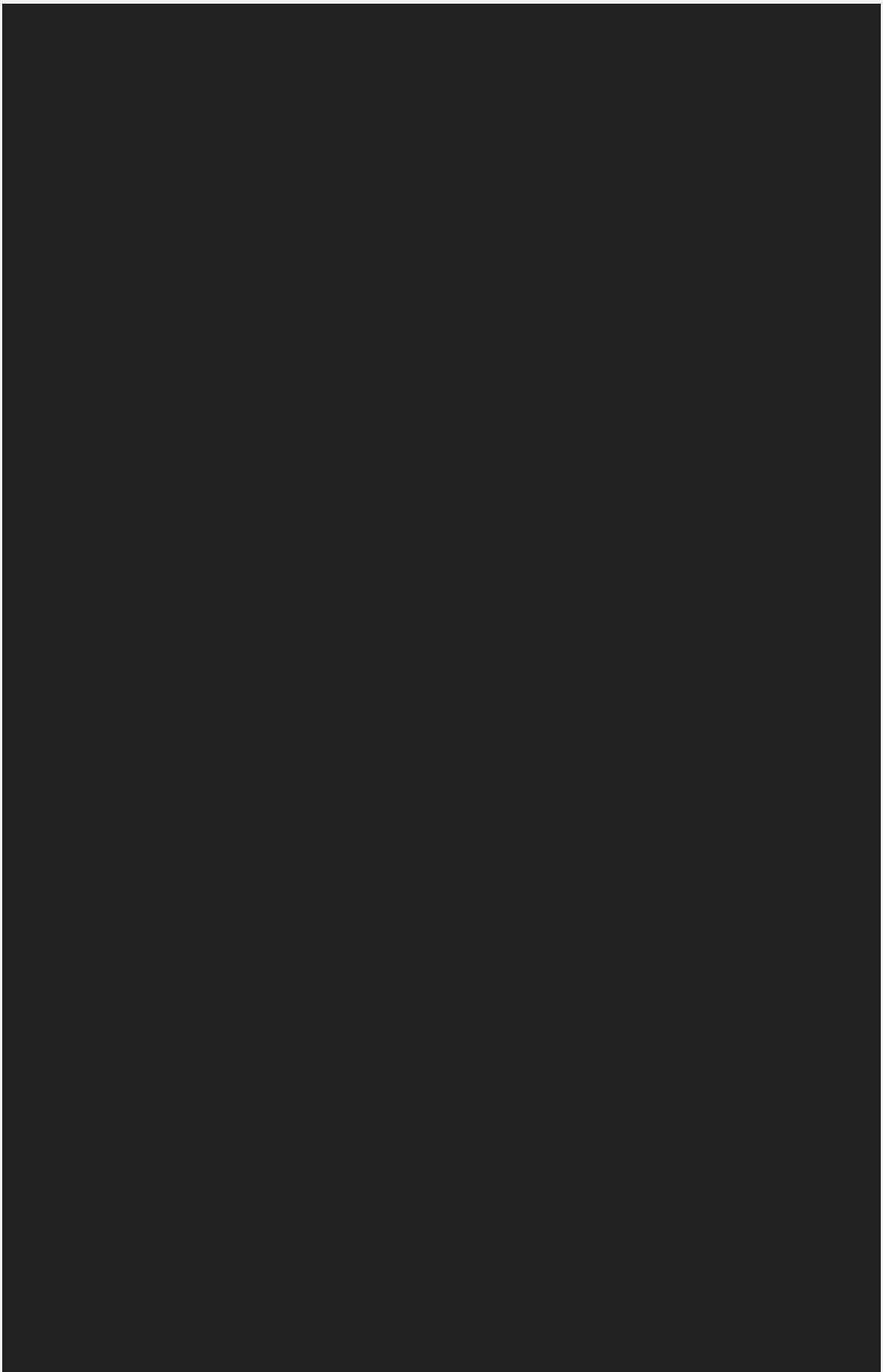
When you compile with `-O2`, GCC performs optimizations such as:

- **Function inlining:** Removes some function-call overhead when profitable.
- **Loop optimization:** Simplifies loops and eliminates unnecessary work where possible.
- **Constant propagation:** Computes expressions at compile time when their values are known.
- **Dead-code elimination:** Removes computations whose results are never used.
- **Instruction optimization:** Chooses more efficient machine instructions.

However, `-O2` does **not** automatically replace your file-reading algorithm with a different one.

In `readAndApplyFilters()`, lines 204–206, you use:

```
None
storage.assign(
    std::istream_iterator<int>(file),
```



This uses formatted stream extraction to convert text into integers.

Even with `-O2`, the implementation still needs to perform integer parsing, handle stream state, and follow the stream's formatting rules.

A manual parser using `std::from_chars` can avoid some of that overhead because it is locale-independent and does not allocate memory during conversion.

That doesn't guarantee it will be faster in your program, but it makes the comparison worthwhile.

3. What measurement would prove an improvement?

I would use an **A/B benchmark** with your existing `StageTimer`.

Compare:

- **Version A:** Original `std::istream_iterator<int>` implementation.
- **Version B:** Alternative buffered reading and parsing implementation.

Compile both with identical `-O2` settings and run each against the same 20 files of 1,000,000 samples.

For more reliable results, alternate A and B runs, perform at least five measured runs of each, and compare their medians.

Reading benchmark calculator

Original median (ms)

Modified median (ms)

Reading time reduction

38.7%

Estimated overall stage speedup

1.54×

Assumes the other measured stages remain at their reported combined 53.77 ms. The initial modified value is hypothetical.

You should also verify correctness. Compare the original and modified programs' standard output using `diff`, and test malformed data, empty files, negative integers, and integer overflow.

One particularly important detail: `std::from_chars` does not automatically skip whitespace or accept leading `+` signs, unlike formatted stream extraction. Your replacement would need to handle those differences if they occur in your `.dat` files.

4. When is the optimization worth keeping?

For a student assignment, I would use these rough guidelines:

preserved

These are judgment guidelines, not universal performance thresholds.

My recommendation: Benchmark a buffered parser as an experiment, but preserve your original implementation until you have evidence. If reading drops from roughly 490 ms to 300 ms while producing identical results, that would be a meaningful improvement. If it drops only to 480 ms, your existing readable implementation may be the better choice.

The most convincing result for your assignment would be a table showing the **median reading time, median total wall-clock time, and percentage improvement**, along with a statement that the program's output remained unchanged.

Explain suggestion <1> as if to a classmate: what C++ feature does it use, why is it better here, and when would it be the

wrong choice? Do not write the code for me.

Suggestion #1 is about replacing `std::istream_iterator<int>` with **buffered file reading and** `std::from_chars`, a C++17 feature that is also available in C++23.

1. What C++ feature does it use?

Right now, my program uses `std::istream_iterator<int>` to read integers directly from a file into a vector.

The alternative uses two features:

- **Buffered reading:** Reads a large chunk of the file into memory instead of extracting integers individually through a stream.
- `std::from_chars`**:** Converts characters representing numbers into actual integers without using the usual stream formatting machinery.

`std::from_chars` is part of the `<charconv>` standard library header.

2. Why is it better here?

My program processes large `.dat` files containing millions of integers separated by whitespace.

`std::istream_iterator<int>` is convenient, but formatted input has extra overhead from stream state management and locale-aware parsing.

`std::from_chars` is more lightweight because it focuses on converting characters into numbers without those additional features.

Since reading takes about **90% of my measured processing time**, even a moderate improvement could significantly reduce the total runtime.

For example, if reading became 25% faster, my overall measured processing time could decrease by approximately 22.5%.

However, I would need to benchmark the change to confirm that it actually improves performance.

3. When would it be the wrong choice?

I wouldn't necessarily use `std::from_chars` for every program.

- **Small files:** The performance difference might be insignificant.
- **Readability:** `std::istream_iterator` requires less code and is easier to understand.
- **Complex input formats:** `std::from_chars` doesn't automatically skip whitespace or handle all the formatting that streams support.
- **Error handling:** I would need to manually handle invalid characters, numbers outside the integer range, and incomplete input.

For a program that only reads a few hundred integers, the simpler stream-based approach would probably be preferable.

4. How would I know it worked?

I would compare the original and modified versions using the same 20 files, compiler settings, and `-O2` optimization.

I would measure the median reading time over five runs and compare the program's output to make sure nothing changed.

In short: `std::from_chars` could be a better choice because my program spends most of its measured time reading large amounts of numeric data. But the extra complexity is only justified if testing shows a meaningful speed improvement without changing the results.